

BIG BROTHER

الأخ الكبير

Current-state assessment and target architecture for the Arabic-first legal self-help platform for Kuwaiti courts: Postgres with row-level security and per-tenant envelope encryption, self-hosted RAG retrieval grounded in the 48-provision Legal Knowledge Base, Counter-Reply v2 with per-sentence traces, and phased delivery under a 162-check compliance suite.

Multi-Tenant SaaS · Kuwait Courts Platform

Next.js 16 · deterministic compliance engine · extends the shipped v2.1 build (38 API routes, 28-rule catalogue, 98-check e2e)

Table of Contents

1. Current-State Assessment	3
1.1 Verified inventory of what exists	3
1.2 Confirmed gaps	5
1.3 PRD-versus-code conflict register	6
2. Target Architecture	7
2.1 Platform decisions and the Postgres migration path	7
2.2 Where the engine stays pure, and where services attach	8
2.3 RAG pipeline service (self-hosted ingestion to indexed store)	8
2.4 Embedding model and vector store — two options each, with tradeoffs	9
2.5 Retrieval API — hybrid, temporally filtered, resolveRef-bound	10
2.6 LLM service — self-hosted, Layer-2 only, trace-per-sentence	10
2.7 Object storage for document originals (S3-compatible, WORM)	11
2.8 OCR / forensics worker	11
2.9 The same diagram as text	11
3. Multi-Tenant Hardening	13
3.1 Per-tenant envelope encryption and KMS design	13
3.2 Defense-in-depth isolation: RLS, middleware, tests	14
3.3 Cross-tenant leakage test suite (added to scripts/e2e.mjs)	14
3.4 Onboarding and offboarding runbook	15
3.5 White-label fields on Organization	15
4. Rule Catalogue Migration	16
4.1 Target: a declarative, versioned rule catalogue	16
4.2 Engine reads the catalogue; behavior stays byte-compatible	17
4.3 What the catalogue unlocks	17
5. Counter-Reply v2	17
5.1 Assertion-extraction pipeline (replaces the regex parser)	17
5.2 Trace IDs and the response table (PRD 5.6 AC3)	18
5.3 Viability rules and amendment re-evaluation	19
6. Safety and Compliance Gates	19
6.1 Minors / domestic-violence / coercion screening at intake	19

- 6.2 Export-blocking rule for unreviewed high-stakes drafts 20
- 6.3 Embedded output-policy text in every pack 20

- 7. API Contracts 20**

- 8. Deliverables and Sequencing 22**
- 8.1 Phase A — Platform: Postgres, RLS, envelope crypto, object storage 22
- 8.2 Phase B — Corpus: RAG ingestion, retrieval, catalogue migration 23
- 8.3 Phase C — Intelligence: counter-reply v2 and the LLM service 23
- 8.4 e2e extension to 162 checks (target 160+) 23

- 9. Decision Register — Only Humans Can Make These 24**

1. Current-State Assessment

This assessment is a verified inventory, not a restatement of documentation. Every row below was checked against the actual source in the repository at commit time: the storage layer (**src/lib/sqlite.mjs** with its typed facade **src/lib/db.ts**), the analysis engine (**src/lib/engine.ts**), the rule catalogue (**src/lib/rules/catalog.ts**), the output builders (**src/lib/packs.ts**, **src/lib/counterReply.ts**), the platform services (**src/lib/billing.ts**, **src/lib/notifications.ts**, **src/lib/arabic.ts**), the 38 route handlers under **src/app/api/****, and the compliance smoke test **scripts/e2e.mjs**. The README claims a 98-check e2e suite; this was verified literally — the file contains exactly 98 **check()** invocations across 11 sections, and the full suite was executed against the running dev server on localhost:3000 with the result **ALL CHECKS PASSED**. Where the PRD (**upload/Big_Brother_PRD_v2.1.md**) and the code disagree, the disagreement is flagged in Section 1.3 instead of being assumed away.

1.1 Verified inventory of what exists

Capability	Verified implementation	Key modules
Multi-tenancy	orgId scoping on every business table (cases, documents, deadlines, drafts, notifications, billing_events); org context derived server-side from the HMAC-signed session cookie on every call; roles OWNER/ADMIN/CASEWORKER/LAWYER/MEMBER; active-org switching with membership check.	src/lib/sqlite.mjs (DDL, FK cascades); src/lib/api-helpers.ts requireOrg(); src/app/api/auth/switch-org/route.ts
Legal Knowledge Base	48 seeded provisions across CPL 38/1980, CrCL 17/1960, Penal 16/1960, E-service 9/2020, Personal Status 51/1996; version chain (version, supersedes_id, effective_from/to, gazette_ref); verified=false default; lawyer-only verify / amend (publishes v+1, supersedes, history retained) / deactivate / reactivate.	prisma/schema.prisma (LegalProvision); src/lib/db.ts verifyProvision(), amendProvision(); src/app/api/kb/audit/**
Citation resolver	Exact then range matching of {lawId, article} against the LKB; unresolved refs carry resolved:false and are never displayed as citations (hallucination guard, PRD 6.3).	src/lib/engine.ts resolveRef(); src/app/api/kb/resolve/route.ts
Deterministic engine	Pure function analyze(CaseData, ProvisionT[]) returning alerts, grounds, deadlines; baseline grounds always surfaced (systematic completeness, PRD 5.5 AC1); no network, no model calls.	src/lib/engine.ts analyze(), buildGround()
Rule catalogue	28 rules hardcoded in TypeScript (9 civil, 5 criminal, 14 family) with predicates, procedural vehicle, timing, severity; appeal/opposition WINDOWS table flagged [JUDGMENT CALL] pending Phase 0.	src/lib/rules/catalog.ts; src/lib/types.ts RuleDef
Two-layer output	Layer 1 educational view is default; Layer 2 draft packs require a non-dismissible acknowledgment at generation (428 when absent) and again at export; draft types carry authority strings ("not for filing").	src/lib/packs.ts DRAFT_TYPES, buildDraftPayload(); src/app/api/cases/[id]/drafts; src/app/api/drafts/[id]/export
Cooling-off + review queue	24h cooling-off for misconduct/nazaha drafts (423 with remainingSec), lawyer-only release; NGO orgs with reviewQueueEnabled route drafts to IN_REVIEW; caseworker release blocked 403.	src/lib/db.ts PackDraft columns; src/app/api/drafts/[id]/finalize release export
Evidence sealing	SHA-256 seal with version chain (previous_id) and mismatch flagging; metaRisk LOW/MEDIUM/HIGH heuristic that never asserts forgery; engine rolls risky docs into AL-FORGERY alerts citing Penal Arts. 257-260.	src/app/api/cases/[id]/documents; src/lib/engine.ts; src/lib/notifications.ts notifyDocumentIntegrity()
Billing and plans	FREE/PRO with capacity limits enforced server-side at 4 choke points; machine-readable 402 plan_limit body; Stripe REST checkout + webhook HMAC verification with event-id idempotency; demo mode without keys; billing ledger with unique ref.	src/lib/billing.ts; src/lib/db.ts setOrgPlan(), getOrgUsage(); src/app/api/billing/**
Notifications	Per-user inboxes (nothing org-wide); dedupe_key idempotency; deadline reminders escalate soon/critical/overdue with new rows per band; event hooks for review requests, releases, hash mismatch, high meta risk.	src/lib/notifications.ts; src/app/api/notifications/**
Arabic search	BM25 over the LKB rebuilt in-memory per request; tokenizer strips the definite article (al-), light suffix stemmer, Arabic-Indic digit folding; fixes the v0.1 recall bug.	src/lib/arabic.ts; src/app/api/search/route.ts
Deadline Radar	deriveDeadlines() computes appeal windows from judgment events and surfaces upcoming sessions; urgency buckets; iCal export endpoint.	src/lib/engine.ts; src/lib/ical.ts; src/app/api/cases/[id]/deadlines-ics

Capability	Verified implementation	Key modules
Audit + data rights	audit() writes audit_logs on every meaningful action; JSON data export; workspace wipe (Section 5.10 workflow); consent records for intake and voice notes.	src/lib/sqlite.mjs audit(); src/app/api/data/export/delete; src/app/api/audit
Compliance e2e	98-check API smoke suite verified passing (health, auth, search fix, resolver, Layer-1, ack gates, cooling-off, review queue, Phase 0 audit, billing, notifications).	scripts/e2e.mjs

Table 1-1 — Verified component inventory (all paths relative to repository root)

1.2 Confirmed gaps

Each gap in the working brief was verified against the code before being accepted. The list below is the confirmed set, with the evidence that justifies each entry. These are gaps relative to the target architecture in Section 2 — none of them is a compliance regression: all existing guardrails (acknowledgment gates, cooling-off clocks, unverified badges, audit trail) are present and tested.

Gap	Evidence in current code	Severity
No vector retrieval	src/app/api/search/route.ts rebuilds a lexical BM25 index in memory on every request over active provisions only; no embedding model or vector store exists anywhere in the stack.	High — semantic recall ceiling
No full-law corpus	Corpus is 48 curated provisions (scripts/seed.mjs, src/lib/seed-data/*.json); rules cite ranges (e.g. 38/1980 arts. 33-42) for which only fragmentary rows exist.	High — blocks Phase 0 scale
No document-original storage	The documents table stores hash, size, mime, meta_risk only; uploaded bytes are hashed and discarded; there is no object reference column.	High — chain-of-custody ceiling
No OCR / forensics worker	No OCR path exists; the only media pipeline is voice-note ASR intake with consent; meta_risk is assigned from user-supplied notes and hash comparisons, not from file analysis.	Medium
Regex-based counter-reply	src/lib/counterReply.ts segments via the ASSERTION_CUE regex and classifies via a 4-pattern Arabic cue list (AR_CUES); template responses per kind; opponent citations are never extracted or validated.	High — PRD 5.6 AC1 shortfall
No per-tenant encryption	One SQLite file at data/big-brother.db (src/lib/sqlite.mjs DatabaseSync); no field-level or envelope encryption; all tenants share one file.	High for SaaS posture
Single SQLite file	Same evidence as above; WAL mode is on, but there is one writer host, no RLS, no point-in-time recovery, no concurrent multi-node reads.	High at production scale
Hardcoded TS rule catalogue	RULES is a compiled TypeScript array in src/lib/rules/catalog.ts; adding a rule requires a redeploy; no validation_status, counterarguments, or viability fields.	Medium — Phase 0 blocker
No interruption/suspension in Deadline Radar	deriveDeadlines() adds fixed WINDOWS[track].appeal days to the judgment date (src/lib/engine.ts); court closures, suspension events, and tolling are not modelled, even though CIV-LIM-004 references interruption of limitation.	Medium — wrong dates risk
No minors/DV safety gate	POST /api/cases validates intake channel and consent only (src/app/api/cases/route.ts); there is no sensitive-case screening or specialist-review routing at intake.	High — safety gap

Table 1-2 — Confirmed gaps with code evidence

1.3 PRD-versus-code conflict register

The working rules for this specification require flagging rather than assuming wherever a PRD claim and code reality conflict. Five such conflicts were identified. They are recorded here, referenced by the sections that resolve them, and carried into the decision register in Section 9 where a human sign-off is required.

ID	PRD claim	Code reality and disposition
C1	PRD 6.3a change management: when a law is amended, existing analyses, alerts, and drafts referencing the superseded version must be flagged for re-evaluation and affected users notified.	amendProvision() in src/lib/db.ts supersedes versions correctly, but no impact scan or notification exists. Resolved in Section 5.3 via response persistence + a re-evaluation job reusing src/lib/notifications.ts.
C2	PRD 5.6 AC3: the point-by-point table includes the user's evidence reference + hash per row.	packs.ts pointByPoint rows carry position, basis, articles, relief, verified — but no per-row evidence hash mapping; hashes appear only on the consolidated integrity page. Partial implementation; resolved in Section 5.2.
C3	PRD success metric: at least 95% of citations resolve to a VERIFIED LKB entry.	verified = 0/48 by design pending Phase 0, so the metric is 0% today. Expected tension, not a defect; tracked by the Phase 0 progress bar (/api/kb/stats).
C4	PRD 6.3a [JUDGMENT CALL] splits "self-hosted": case data needs region-compliant private hosting; the legal-content layer is a licensing problem and need not be self-hosted.	The working brief for this specification mandates all new services self-hosted with no case data leaving the deployment. This spec adopts the stricter reading; the PRD's looser split remains open for human confirmation (Appendix E-2).
C5	PRD 5.6 AC2: response categories are admission / qualified admission / denial.	buildResponse() in src/lib/counterReply.ts never offers admission — the system never suggests admitting the opponent's allegation. Intentional conservative deviation; kept, and recorded as a product decision.
C6	PRD Appendix E-9: appeal windows are judgment calls pending lawyer validation.	WINDOWS in src/lib/rules/catalog.ts hardcodes civil 40 / criminal 20 / family 30 days; the family window is labeled "under verification" in the engine's own alert strings. Must be resolved by the Phase 0 audit; Section 4.3 moves windows into the versioned catalogue.

Table 1-3 — Flagged conflicts between PRD v2.1 claims and code reality

Two claims in the working brief itself were also verified rather than assumed. The characterization of the counter-reply parser as regex-based is accurate (see Table 1-2). The figure of 98 e2e checks is accurate and all 98 pass; the target in Section 8 grows the suite to 162 checks without changing the semantics of the existing 98.

2. Target Architecture

2.1 Platform decisions and the Postgres migration path

Next.js 16 App Router with TypeScript remains the application platform; nothing in the current route-handler architecture (`src/app/api/**`, 38 handlers) needs to change shape. The decisive platform change is storage: Postgres 16 becomes the production datastore, while node:sqlite stays the developer default. The migration is justified by four concrete deficits verified in Section 1.2, none of which SQLite can fix: (a) row-level security for defense-in-depth tenant isolation (Section 3.2) — SQLite has no equivalent; (b) concurrent multi-node reads and writes for a multi-tenant SaaS with NGO and law-firm workspaces; (c) pgvector for the retrieval service (Section 2.4); and (d) point-in-time recovery and managed backups, which a single WAL-mode file cannot provide.

The migration path preserves the existing seams. **src/lib/db.ts** is already the only typed facade over the storage layer, and **prisma/schema.prisma** already documents every model in Prisma form as a portability statement (its header says exactly this). The path is therefore: (1) keep the DDL in **src/lib/sqlite.mjs** as the dev schema and mirror each new table in **prisma/schema.prisma**; (2) introduce a `StorageAdapter` interface with two implementations — `SqliteAdapter` (current SQL, unchanged behavior) and `PostgresAdapter` (parameterized SQL against Postgres with RLS session variables set per transaction); (3) select the adapter by environment (`DATABASE_DRIVER=sqlite|postgres`), so dev keeps running with zero services and CI runs the same 98-check suite against both drivers; (4) migrate data with a one-shot copy script that maps the `snake_case` DDL of `sqlite.mjs` onto the Postgres schema. No route handler changes its imports; only `db.ts` internals change.

2.2 Where the engine stays pure, and where services attach

The current engine boundary is the most valuable architectural property in the codebase and it is preserved verbatim: **src/lib/engine.ts** exports `analyze(c: CaseData, provisions: ProvisionT[])` — a pure function with no I/O, no clock dependence beyond its explicit `today()` input surface, and no model calls. Layer 1 output is deterministic and reproducible, which is what makes the 98-check suite stable and the compliance posture auditable. This specification does not move the engine.

New services attach strictly at the API layer, feeding the engine or the Layer-2 draft builders with material that has already been resolved and verified: the Retrieval API supplies candidate provision IDs that are passed through the existing `resolveRef()` semantics before display; the LLM service produces Layer-2 draft language whose every sentence carries a trace (Section 2.6); the forensics worker enriches document metadata that the engine already consumes through the `metaRisk` field (the `AL-FORGERY` rollup in `engine.ts` reads `d.metaRisk`). Rule text flows inward (public corpus), case facts never flow outward to third parties (tenant-scoped), which is the data-sensitivity boundary drawn in Figure 2-1.

2.3 RAG pipeline service (self-hosted ingestion to indexed store)

A dedicated ingestion service owns the path from official sources to the indexed store. Sources are the Kuwait Official Gazette (Kuwait Al-Youm — the gazette reference format already recorded on provisions, e.g. the e2e fixture "Kuwait Al-Youm, 1996 supplement") and the Ministry of Justice legal portal, subject to the licensing gate the PRD already imposes (Section 9.2: "no bulk indexing of restricted MOJ source text without a resolved licensing/data-use position"). The PRD's manual-entry-first pattern remains the fallback and the Phase 0 path; the pipeline below is the scale path once licensing is signed.

1. **Acquisition:** scheduled fetchers download gazette issues and MOJ legislation pages to a staging bucket; each artifact is content-hashed (SHA-256, reusing the hashing convention of `src/lib/sqlite.mjs`) and recorded in an `ingest_jobs` table with source URL, issue date, and license note.
2. **Arabic OCR:** scanned pages pass through the OCR/forensics worker (Section 2.8) — Tesseract ara traineddata as the baseline, with an optional layout-aware model — producing text with page coordinates; confidence below threshold is flagged for human review rather than silently indexed.
3. **Article-anchored chunking:** the chunker segments law text at article boundaries — one chunk per article (or per bounded article range), each chunk carrying `law_id`, `article_no`, `version`, `effective_from`,

effective_to, amendment_version, and the heading trail (book/part/section) as inherited context. This preserves exactly the fields the LegalProvision table already models (prisma/schema.prisma), so a chunk resolves 1:1 onto an LKB row.

- 4. **Amendment linkage:** amendment articles ("Article X is replaced by...") create supersedes edges between chunk versions, mirroring the supersedes_id chain that amendProvision() already maintains for the LKB; the chunk store never overwrites history, matching PRD 6.3a.
- 5. **Embedding and indexing:** chunks are embedded (Section 2.4) and written to the vector store together with their lexical representation; indexing is idempotent per content hash.

2.4 Embedding model and vector store — two options each, with tradeoffs

The PRD constrains the choice: retrieval is a matching aid only (6.3a), candidates must resolve to exact LKB entries, and hosting must keep case data inside the deployment. Both options below are open-weight and self-hostable; both are Arabic-capable; the tradeoff is retrieval quality versus serving weight.

Option	Profile	Strengths	Tradeoffs
BAAI/bge-m3 (MIT)	568M params, 1024-dim dense + learned sparse + ColBERT multi-vector; multilingual with strong Arabic retrieval results.	One model yields dense and sparse signals (natural fit for hybrid retrieval); best Arabic recall of the two; single container for the whole indexing pipeline.	Largest memory footprint (~2.3 GB fp32); GPU recommended for batch indexing throughput; multi-vector mode needs extra storage if enabled (can be disabled).
intfloat/multilingual-e5-large (MIT)	560M params, 1024-dim dense only; XLM-R backbone with query:/passage: prefix convention.	Simplest serving (ONNX int8 quantizes well for CPU-only deployments); long-standing, well-understood behavior; slightly lower latency.	Dense-only — Arabic keyword fidelity depends entirely on the separate BM25 leg; generally a step behind bge-m3 on Arabic retrieval benchmarks; prefix convention must be enforced in code.

Table 2-1 — Self-hostable Arabic-capable embedding options

Recommendation: bge-m3 where a GPU (even a single 24 GB card) is budgeted, because its sparse leg strengthens Arabic legal-term matching alongside BM25; multilingual-e5-large where the deployment is CPU-only. The vector store has the same two-option shape: **pgvector** inside the Phase A Postgres (HNSW indexes, transactional with the RLS-scoped data, one fewer service to run, pre-filtering on isActive/effective dates is a plain SQL WHERE) versus **Qdrant** (purpose-built hybrid search with payload filters, easier horizontal scaling, but a second stateful service and a synchronization burden with the LKB). Recommendation: pgvector first — it matches the Phase A platform and the corpus size (thousands to low millions of chunks); move to Qdrant only if corpus or QPS outgrows it. Both choices are listed in the human decision register (Section 9).

2.5 Retrieval API — hybrid, temporally filtered, resolveRef-bound

The Retrieval API (contract in Section 7) exposes hybrid search: BM25 over the chunk store and kNN over vectors, reciprocal-rank fused. Two properties are non-negotiable. First, **temporal validity**: every query carries an as-of instant (defaulting to now) and the store filters isActive = true AND effective_from less-or-equal as-of AND (effective_to IS NULL OR effective_to greater-than as-of), so superseded articles are never retrieved for current-law questions; historical questions may opt in by passing an explicit as-of date. This closes the gap flagged in Table 1-2 — today the search route filters only is_active (src/app/api/search/route.ts calls getAllActiveProvisions()), while effective_from/effective_to are recorded but unused in retrieval. Second, **resolution discipline**: the API returns provision IDs plus scores; it never renders text as a citation. Consumers pass candidate IDs through the existing resolveRef() (src/lib/engine.ts) — the exact semantics of /api/kb/resolve — so a retrieval hit that fails to resolve to a current LKB row is discarded or badged "unverified", never displayed. This is the PRD 6.3a rule that retrieval output is never shown directly as a citation.

The Arabic tokenizer of **src/lib/arabic.ts** is reused as the lexical leg's analyzer (definite-article strip, light stemmer, digit folding), which keeps query normalization identical between the legacy in-app search and the new retrieval service — the v0.1 bare-form/definite-form recall fix must not regress when the corpus grows a thousandfold.

2.6 LLM service — self-hosted, Layer-2 only, trace-per-sentence

The LLM service is self-hosted and open-weight, served behind an OpenAI-compatible API by vLLM on the deployment's own GPU. Recommended primary: **Qwen2.5-32B-Instruct** (Apache-2.0; strong Arabic generation; fits a single 40-48 GB GPU with AWQ quantization, or two 24 GB cards) with a documented scale path to Qwen2.5-72B-Instruct when hardware allows; a lighter Arabic-specialist alternative (e.g. jais-family) is viable for CPU-constrained deployments. Final model selection, GPU budget, and license verification are human decisions (Section 9). The service is used **ONLY** for Layer-2 discussion-draft language — Layer 1 remains the deterministic engine — and only with retrieval-grounded prompting: the prompt contains resolved provision text from the LKB, never free web text.

Every generated sentence must carry a trace before it can render: the draft builder emits per-sentence records {span, provision_id, evidence_hash, extracted_text} or marks the sentence verified:false, and the UI renders unverified sentences with the existing unverified badge convention (the ResolvedCitation label machinery in engine.ts and the pack payload). External LLM APIs are permitted only for anonymized rule text — never case facts — which is the strict reading of PRD 6.3a's hosting clause adopted in C4.

2.7 Object storage for document originals (S3-compatible, WORM)

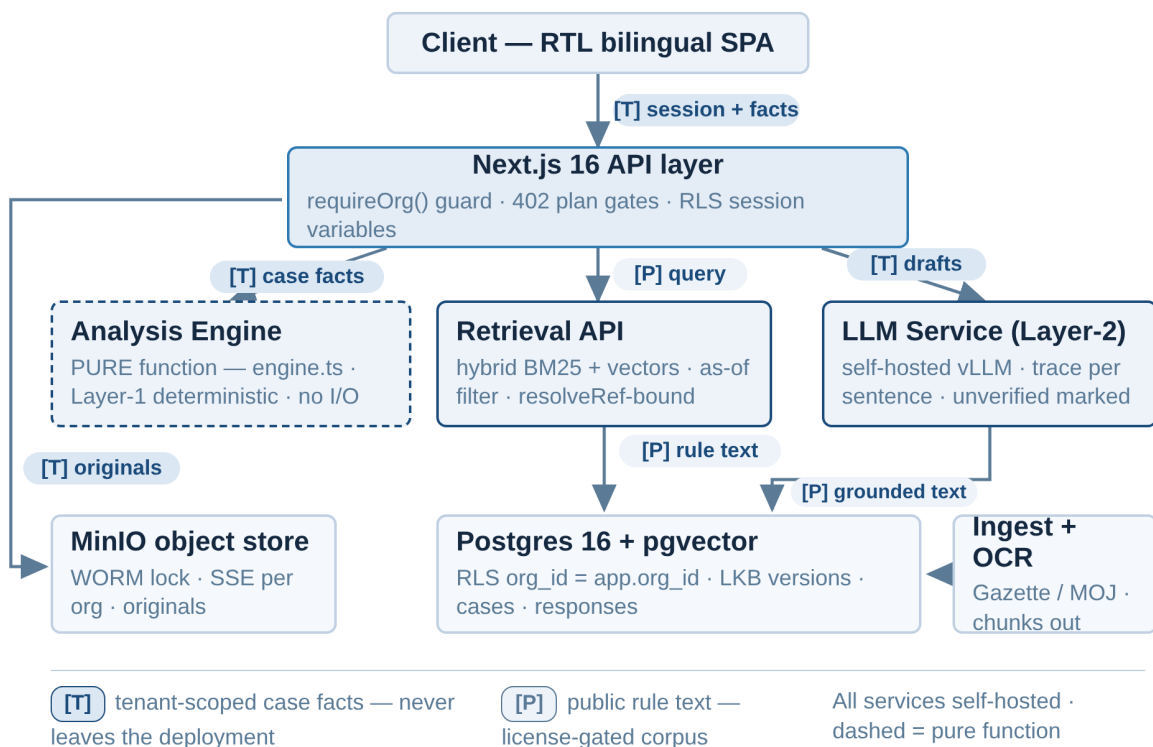
A self-hosted MinIO cluster provides S3-compatible object storage with versioning and object-lock. Bucket layout mirrors tenancy: `bb/org/{orgId}/case/{caseId}/doc/{docId}/v{n}`, with object-lock (COMPLIANCE mode) applied to sealed evidence objects so originals are immutable for the retention period, and server-side encryption wired to the per-org envelope keys of Section 3.1. Integration with the existing hash chain is additive: the documents route (`src/app/api/cases/{id}/documents`) gains a multipart mode that (1) computes SHA-256 server-side exactly as today, (2) writes the object, (3) stores `object_key` alongside the existing `hash/size/mime/meta_risk/version/previous_id` columns, and (4) keeps the mismatch flagging and `notifyDocumentIntegrity()` behavior unchanged. Hash-only rows remain valid for documents that never upload originals, preserving backward compatibility with the 98-check suite. The PRD 5.10 deletion workflow reconciles as follows: workspace deletion starts the existing 7-day window; at completion, objects are deleted (versioned delete markers) while the hash-only seal records remain in the audit trail as non-personal evidence that sealing occurred.

2.8 OCR / forensics worker

A queue-driven worker (pg-boss on the Phase A Postgres, avoiding an extra broker service) consumes uploaded originals and produces structured metadata: EXIF-style creation/modification dates, producer software, page count, and OCR text. It then runs **timeline contradiction checks** — comparing file timestamps and in-document dates against the case timeline (`case_events`, which `deriveDeadlines()` already orders by date) — and proposes `metaRisk` upgrades through the same LOW/MEDIUM/HIGH vocabulary the documents table already stores. Outputs feed `metaRisk` only; the worker never emits forgery conclusions, matching the engine's existing language ("we do not conclude forgery — we flag irregularity risk", `engine.ts` AL-FORGERY alert) and PRD 5.4 AC3. All findings attach to the document's evidence hash so every forensic claim is traceable.

2.9 The same diagram as text

This section gives the explicit text form of the architecture requested for this specification, followed by the rendered diagram. Arrow labels carry the data-sensitivity class used throughout this document: [T] tenant-scoped case facts, [P] public rule text.



Technical Architecture Specification - v1.0

3. Multi-Tenant Hardening

3.1 Per-tenant envelope encryption and KMS design

Today all tenants share one SQLite file with no encryption (Table 1-2). The target design is envelope encryption with a key hierarchy: a root master key held by a self-hosted KMS; per-organization data encryption keys (DEKs); and ciphertext at the field and object level. The recommended KMS is **HashiCorp Vault Transit** (self-hostable, audited key lifecycle, no plaintext keys ever leave the service); for developer machines a SoftHSM backend behind the same interface suffices. The application never sees a master key — it asks the KMS to unwrap a stored DEK inside the request path, caching unwrapped DEKs in memory for the process lifetime only.

Scope matters as much as mechanism. Encrypted under the org DEK: case narrative fields (cases.opponent_pleading, event notes, subject_name), voice-note transcripts, document originals (via MinIO SSE with per-org keys), and pack payloads. NOT encrypted: identifiers, org_id, timestamps, status fields — these must remain queryable for RLS, indexing, and the usage metering already implemented in getOrgUsage() (src/lib/db.ts). A new org_keys table (key_id, org_id, wrapped_dek, algo, state, created_at, rotated_at) records the hierarchy; every key operation is written to the audit trail via the existing audit() helper in src/lib/sqlite.mjs.

Rotation is cheap by construction: rotating an org key wraps a new DEK version and re-wraps the old one (a metadata operation); re-encryption of data proceeds lazily on write and via a background job, with the old DEK held in state "retired" until the job drains. **Crypto-shredding** on offboarding destroys the org's DEKs, rendering every remaining ciphertext permanently unreadable even from backups. This must be reconciled with two existing obligations. First, the 7-day deletion workflow (PRD 5.10, implemented in src/app/api/data/delete): the shredding job runs only after the 7-day grace window completes and rows are purged, so a cancelled deletion still finds its data readable. Second, the immutable audit trail: audit_logs are deliberately kept OUTSIDE the envelope — they contain pseudonymous ids, actions, and detail strings rather than case content, and they are encrypted under a separate platform audit key that is never shredded. Without this split, crypto-shredding an org would silently destroy the compliance history that PRD 6.7 requires to survive. This reconciliation decision is flagged for counsel sign-off in Section 9.

3.2 Defense-in-depth isolation: RLS, middleware, tests

Isolation today rests on a single layer: `requireOrg()` in `src/lib/api-helpers.ts` derives the session, looks up the org, checks membership, and every query hand-writes an `org_id` predicate. That is correct but fragile — one forgotten `WHERE` clause is a breach. The target adds two more layers around the same interface. Layer 1 (application guard) stays: `requireOrg()` is unchanged, and a shared query helper makes org-scoped predicates the default rather than the discipline. Layer 2 (database) is Postgres RLS: tenant tables enable `ROW LEVEL SECURITY` with a policy of the form `org_id = current_setting('app.org_id', true)`; the `PostgresAdapter` opens each request transaction with `SET LOCAL app.org_id`, so a forgotten `WHERE` can no longer leak rows. Migrations run under a separate `BYPASSRLS` role that the application never uses. Layer 3 (verification) is the cross-tenant leakage suite of Section 3.3, executed in CI against the Postgres driver.

3.3 Cross-tenant leakage test suite (added to `scripts/e2e.mjs`)

The suite below extends the existing e2e patterns — two workspaces are created, the attacker logs into workspace A and probes resources owned by workspace B using forged ids, switched sessions, and enumeration. The 22 checks are numbered for direct implementation; each asserts the negative result (404/403/401, or zero rows) and, where relevant, that the audit log recorded the attempt.

1. T-01 `GET /api/cases/{B-case-id}` from A returns 404 and no field leakage in the error body.
2. T-02 `PATCH /api/cases/{B-case-id}` from A returns 404; B-side row unchanged afterwards.
3. T-03 `GET /api/cases/{B-case-id}/analysis` from A returns 404 and no `analysis.run` audit row for B.
4. T-04 `GET /api/cases/{B-case-id}/events` from A returns 404.
5. T-05 `POST` event into B case from A returns 404.
6. T-06 `POST` document seal into B case from A returns 404; no documents row created.
7. T-07 `GET /api/drafts/{B-draft-id}` from A returns 404.
8. T-08 `POST /api/drafts/{B-draft-id}/export` from A returns 404.
9. T-09 `POST /api/drafts/{B-draft-id}/release` from A (even with lawyer role in A) returns 404.
10. T-10 `GET /api/cases/{B-case-id}/deadlines-ics` from A returns 404.
11. T-11 `POST /api/notifications {id: B-notification-id}` from A marks 0 rows (recipient-scoped update).
12. T-12 `GET /api/notifications` from A never returns rows whose `orgId` belongs to B.
13. T-13 `POST /api/auth/switch-org {orgId: B-org-id}` without membership returns 403 and leaves the active org unchanged.
14. T-14 `POST /api/auth/switch-org {joinCode: forged-guess}` returns 4xx; 10 rapid guesses all fail consistently (enumeration resistance).
15. T-15 Tampered session cookie (payload byte flipped, HMAC invalid) yields 401 on every org-scoped route.
16. T-16 Session cookie re-signed for another uid (key unknown to attacker — simulated by replaying A's cookie against B's resources) still yields 404s; no id-based access.

17. T-17 MEMBER role POST /api/billing/checkout in own org returns 403 (privilege escalation resistance).
18. T-18 MEMBER POST /api/kb/audit/{id} verify returns 403 (role gate holds across orgs).
19. T-19 GET /api/data/export from A returns only A-org cases; row count equals A's case count.
20. T-20 GET /api/audit from A returns only A-org log rows.
21. T-21 (Postgres driver) direct SQL under the app role with app.org_id=A cannot SELECT B rows (RLS unit test in CI).
22. T-22 (Postgres driver) unsetting app.org_id in-session yields zero tenant rows (no silent fallback to superuser semantics).

3.4 Onboarding and offboarding runbook

Phase	Steps	Controls
Onboard	1. Create organization (POST /api/orgs) — type, plan, reviewQueueEnabled. 2. Issue join code (auto). 3. Owner membership. 4. Provision org DEK in KMS. 5. Create MinIO prefix bb/org/{orgId} with object-lock default. 6. Set white-label fields. 7. Seed RLS binding (org row exists).	Join-code entropy check; DEK creation logged; storage prefix policy lock verified; first-login acceptance of consent version (consents table).
Operate	Plan changes via /api/billing (ledger + webhook idempotency); membership changes restricted to OWNER/ADMIN; usage meters from getOrgUsage().	402 plan_limit convention preserved; safety features never gated (billing.ts design note).
Offboard	1. Owner requests workspace deletion (mode: workspace). 2. 7-day grace: login blocked for members, export available to owner. 3. Day 7: purge rows, delete objects (versioned), release MinIO prefix. 4. Crypto-shred org DEKs in KMS. 5. Tombstone org row (slug reserved).	Shred only after purge completes; audit rows survive (kept under platform audit key); join code invalidated; webhook-driven cancellations reconciled via billing ledger refs.
Restore	Within grace window: owner cancel restores access; keys and objects untouched.	Single audit entry, no partial state (transactional in PostgresAdapter).

Table 3-1 — Tenant lifecycle runbook (each step audited via audit())

3.5 White-label fields on Organization

The organizations table (src/lib/sqlite.mjs DDL; prisma/schema.prisma Organization model) gains presentation fields so NGOs and law firms can brand their workspace without code changes: display_name, logo_url, primary_color, default_locale (ar default), custom_domain, footer_text_ar, footer_text_en, theme_json. They are surfaced read-only through /api/auth/me (activeOrg payload) and editable via an OWNER/ADMIN-gated PATCH on /api/orgs. Custom domains resolve through a host-header middleware that looks up the org by custom_domain and pins the session's allowed orgs — a thin extension of the existing switch-org trust model. All fields are tenant-scoped data and therefore inside the envelope-encryption scope decision of Section 3.1 (logo and colors are not sensitive; they are excluded for queryability, which the spec notes explicitly to keep the encryption scope list honest).

4. Rule Catalogue Migration

4.1 Target: a declarative, versioned rule catalogue

The 28 rules currently live as a compiled TypeScript array (src/lib/rules/catalog.ts: RULES, plus LAW, WINDOWS, FORGERY_RULE, EVENT_TYPES). Every addition or Phase 0 correction requires a redeploy, and the catalogue lacks fields the brief requires — counterarguments, interruption/suspension rules, viability scoring, and validation metadata. The target is a declarative catalogue stored in Postgres (mirrored in the dev SQLite DDL) and validated by a schema on load. The requested fields map onto the following table definition; existing RuleDef fields are carried over 1:1 so nothing is lost in translation.

RULE CATALOGUE – TABLE DDL (Postgres; SQLite mirror in sqlite.mjs)

```
CREATE TABLE rule_catalogue (
  rule_id          TEXT PRIMARY KEY,          -- 'CIV-SERV-005'
  law_id           TEXT NOT NULL,             -- '38/1980'
  track            TEXT NOT NULL,             -- civil | criminal | family
  category         TEXT NOT NULL,             -- service | limitation | ...
  statement_ar     TEXT NOT NULL,             -- from RuleDef.ar
  statement_en     TEXT NOT NULL,             -- from RuleDef.en
  display_articles JSONB NOT NULL,             -- from RuleDef.articles
  article_refs     JSONB NOT NULL,             -- [{lawId, article}] for resolveRef
  required_predicates JSONB NOT NULL,         -- from RuleDef.predicates
  procedural_vehicle JSONB NOT NULL,           -- {ar, en}
  timing_constraints JSONB NOT NULL,          -- {ar, en} + machine fields
  interruption_suspension JSONB NOT NULL DEFAULT '[]',
  counterarguments JSONB NOT NULL DEFAULT '[]',
  viability_scoring JSONB NOT NULL DEFAULT '{}',
  default_severity TEXT NOT NULL,             -- high | medium | low
  validation_status TEXT NOT NULL DEFAULT 'pending', -- Phase 0 states
  validator        TEXT,                     -- lawyer id|name stamp
  validated_at     TEXT,
  version          INTEGER NOT NULL DEFAULT 1,
  supersedes_id   TEXT,
  is_active        BOOLEAN NOT NULL DEFAULT TRUE,
  created_at       TEXT NOT NULL,
  updated_at       TEXT NOT NULL,
  UNIQUE (rule_id, version)
);
-- WINDOWS moves here too: (track, kind, days, validation_status, validator,
-- validated_at, version) so hardcoded appeal windows become auditable data.
```

The JSON Schema mirror of this table lives beside the code (rules/schema.json) and is enforced by the loader with zod-style validation: an invalid rule (missing predicate text, unknown track, article_refs failing LKB shape) fails closed at startup in dev and is quarantined in prod, never silently ignored. The catalogue version chain reuses the pattern amendProvision() already established for the LKB — new version inserted, old row retired, history never overwritten (PRD 6.3a change management).

4.2 Engine reads the catalogue; behavior stays byte-compatible

A RuleProvider interface (list(track), get(rule_id)) is injected into analyze(c, provisions, rules) with a default of the current compiled RULES, so the existing signature remains source-compatible for every caller (src/app/api/cases/[id]/analysis/route.ts passes the provider; tests pass fixtures). The migration script dumps the current 28 rules to seed JSON (exact TS-to-JSON field mapping: id to rule_id, ar/en to statement_ar/en, articles to display_articles, refs to article_refs, predicates to required_predicates, vehicle to procedural_vehicle, timing to timing_constraints, severity to default_severity) and inserts them at version 1 with validation_status="pending" — honest Phase 0 state. A feature flag RULES_SOURCE=db|ts keeps both paths available until parity is proven.

Parity is defined mechanically: five fixture cases (the two seeded demo cases plus three synthetic civil/criminal/family timelines) produce golden-file analyze() outputs — alerts, grounds, deadlines byte-identical pre- and post-migration; the existing 98 e2e checks must pass unchanged, which the brief explicitly requires. New catalogue fields start empty and are populated during Phase B lawyer validation (validation pipeline is a human decision, Section 9); the engine ignores them until present, so enabling the DB source cannot change output.

4.3 What the catalogue unlocks

Three sections of this specification depend on this migration. Deadline Radar v2 reads interruption_suspension to toll or suspend windows (court recess, service suspension, adjournment-linked pauses), replacing the fixed WINDOWS addition in deriveDeadlines() — the confirmed gap of Table 1-2 — while keeping the "verify with a lawyer" framing on every derived deadline. Counter-Reply v2 (Section 5) reads required_predicates for the automatic viability downgrade and counterarguments to render educational "what the other side may argue" context. The Phase 0 audit gains a work surface: validation_status/validator/validated_at turn rule verification into the same lawyer-signed, gazette-referenced workflow the LKB already has in /api/kb/audit.

5. Counter-Reply v2

5.1 Assertion-extraction pipeline (replaces the regex parser)

The current parser (src/lib/counterReply.ts) splits on punctuation and classifies by four Arabic cue regexes; it neither extracts the opponent's citations nor scores materiality, and its responses are templates keyed by class. Counter-Reply v2 replaces this with a five-stage pipeline in which the regex stage survives only as the deterministic Layer-1 fallback, preserving the existing educational behavior when no model is configured.

1. **Segmentation:** Arabic-aware sentence segmentation (enders: period, exclamation, the Arabic question mark, semicolon, newline; plus connector-word heuristics) producing atomic assertions; the current ASSERTION_CUE split becomes the degenerate case.
2. **Classification:** assertion classes remain the stable five — allegation, citation, relief, evidence, fact (the AR_CUES vocabulary) — each with a confidence; cue-based classification is the floor, an optional local

classifier the floor when configured.

3. **Materiality scoring:** a deterministic score per assertion (class weight, severity of any referenced rule, track weighting) that orders the point-by-point table and drives which points are summarized in Layer-1; scoring inputs stay inside the case workspace.
4. **Opponent-citation extraction:** legal-reference patterns for Arabic pleadings (article-number forms: "m" + number, "law no. x/year" constructions, decree/regulation citations, range citations) are extracted as candidate {lawId, article} pairs with verbatim spans.
5. **Validation against the LKB:** every extracted pair passes through the existing resolveRef() semantics (src/lib/engine.ts; /api/kb/resolve): resolved+verified renders with its badge; resolved+unverified renders badged pending Phase 0; unresolved is never displayed as a citation. A resolved citation whose article range or text contradicts the opponent's claimed content is explicitly flagged "misstated provision — the cited text does not match the current official text" in neutral educational language; the system never asserts that the opponent lied (UPL boundary, PRD 9.3).

5.2 Trace IDs and the response table (PRD 5.6 AC3)

Every response line gets a trace_id at generation time, and persistence makes the AC3 table queryable rather than a transient payload. Two tables, mirrored into prisma/schema.prisma:

COUNTER-REPLY V2 — RESPONSE AND TRACE SCHEMA

```
CREATE TABLE counter_responses (
  id          TEXT PRIMARY KEY,
  org_id      TEXT NOT NULL REFERENCES organizations(id),
  case_id     TEXT NOT NULL REFERENCES cases(id),
  assertion_id INTEGER NOT NULL,      -- point number in the pleading
  point_text  TEXT NOT NULL,         -- opponent's point (verbatim span)
  kind        TEXT NOT NULL,         -- allegation|citation|relief|evidence|fact
  materiality REAL NOT NULL DEFAULT 0,
  position    TEXT NOT NULL,         -- response category (bilingual JSON)
  response_text TEXT NOT NULL,       -- bilingual JSON {ar, en}
  legal_basis JSONB NOT NULL,        -- [{provision_id, ref, verified, label}]
  evidence_refs JSONB NOT NULL,      -- [{doc_id, hash, name}] per AC3
  trace_id    TEXT NOT NULL,
  layer       TEXT NOT NULL,         -- L1 | L2
  needs_human_review BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TEXT NOT NULL
);

CREATE TABLE response_traces (
  trace_id    TEXT NOT NULL,
  response_id TEXT NOT NULL REFERENCES counter_responses(id),
  kind        TEXT NOT NULL,         -- extracted_text | provision | evidence_hash
  provision_id TEXT,                 -- when kind = provision
  evidence_hash TEXT,               -- when kind = evidence_hash
  span        TEXT,                 -- verbatim source span
  created_at  TEXT NOT NULL
);
```

This closes conflict C2: the point-by-point payload of buildDraftPayload() (src/lib/packs.ts) is populated from counter_responses rows, so each point carries its own evidence reference + hash exactly as AC3 specifies, while the consolidated integrity page remains for the whole pack. The response layer also creates the substrate for conflict C1: rows record the provision ids they rely on, which is what the amendment re-evaluation job of Section 5.3 scans.

5.3 Viability rules and amendment re-evaluation

Viability becomes mechanical. The catalogue's required_predicates (Section 4) are matched against the satisfied-predicate evidence already computed by buildGround() (engine.ts passes satisfiedPredicates into GroundItem today). When required predicates are missing, relevance downgrades automatically one band (high to medium to low) with a bilingual explanation of which predicate is unmet — extending, not replacing, the existing "relevance is not a legal determination" framing (types.ts GroundItem comment). A dismissal-relevant item may only render at high relevance when all required predicates are satisfied OR when needs_human_review is forced true, which renders the item behind the mandatory "needs human lawyer review" state — the existing needsReview flag on GroundItem, now load-bearing.

Amendment re-evaluation closes conflict C1 end to end. When amendProvision() publishes version v+1 and retires v (src/lib/db.ts), a job scans counter_responses.legal_basis and persisted ground citations for the retired provision id, marks the affected rows re-evaluate=true, and notifies the owning users through the existing notifyUser() machinery (src/lib/notifications.ts) with a dedupe key of the form provision:{id};amended — idempotent by the same unique index that powers deadline reminders. Previously viewed analyses and drafts are thereby flagged exactly as PRD 6.3a requires, without any retroactive mutation of history.

6. Safety and Compliance Gates

6.1 Minors / domestic-violence / coercion screening at intake

Intake today records consent and intake channel but performs no sensitive-case screening (src/app/api/cases/route.ts validates intakeChannel, consentText, subjectName only). The gate adds a deterministic screen at case creation over the signals the request already carries: subType and event hints, and keyword classes over free text (note, opponentPleading, subjectName context) covering minors (guardianship, infancy, and infant-custody markers), domestic violence (family-violence, protection, and restraint markers), and coercion (duress, threats, and impersonation markers). On detection the case is flagged sensitive_case with a class enum, and one of two dispositions applies per organization configuration: (a) **specialist-review routing** — the case enters the org's review workflow and reviewers with the LAWYER role are notified at creation, reusing the fan-out pattern and per-user inbox of notifyReviewersOfDraft() in src/lib/notifications.ts; or (b) **constraint mode** — the engine runs but relevance is capped at low, Layer-2 generation for counter/dismissal types is disabled, and every surface shows the existing "consult a lawyer" framing with an explicit "this case type needs specialist support" notice. The gate never blocks data entry or sealing (evidence preservation is a safety feature) and never produces a diagnosis; both dispositions are educational and route the user toward qualified help. NGO partners can be granted the specialist queue by the existing reviewQueueEnabled flag, which keeps the feature inside the current tenancy model.

6.2 Export-blocking rule for unreviewed high-stakes drafts

The export path (`src/app/api/drafts/[id]/export`) already enforces four gates: acknowledgment (428), finalize-before-export (409), cooling-off (423 with `remainingSec`), and the `IN_REVIEW` lock for review-queue orgs. One rule is added, none removed: **High-relevance dismissal grounds and misconduct/nazaha drafts cannot export unless lawyer-reviewed (status RELEASED) or the export is explicitly labeled DRAFT.** The unlabeled path returns 423 with a machine-readable body `{reason: "lawyer_review_required", reviewRequired: true}`; the labeled path proceeds only with a DRAFT watermark and the not-for-filing authority string stamped into the exported document and the acknowledgments record (which already snapshots banner text per export). This composes with the viability rule of Section 5.3: a dismissal draft whose high-relevance grounds lack required predicates is blocked from the unlabeled export regardless of draft age. All existing gates are preserved verbatim — acknowledgment at generation and export, 24h cooling-off clocks, review-queue release restrictions — and the 98-check suite already covers their absence paths.

6.3 Embedded output-policy text in every pack

The pack builders (`src/lib/packs.ts`) already embed the mandatory cover notice (`draftCoverBilingual()`), the relief-category disclaimer ("general information, NOT ready-to-file requests"), and registry information framed as public knowledge. The policy layer is unified into a single `OUTPUT_POLICY` block — bilingual, versioned like `bannerVersion`, and rendered at three positions: the cover, a footer line on every exported section, and the acknowledgment snapshot. The block states, at minimum: the document is educational; it is not a conclusion that any document is forged, any party is guilty, any professional committed misconduct, or any claim will be dismissed; citations marked unverified are pending lawyer audit; and no part of the document may be filed without independent legal review. Embedding the policy in the payload (rather than only the UI shell) means exports carry the policy even when rendered outside the app, which the PRD's export-time acknowledgment requirement implies.

7. API Contracts

The sketches below follow the conventions already established in the codebase: cookie session auth via `requireOrg()` (`src/lib/api/helpers.ts`), org scoping derived server-side, the error shape `{error: string, ...context}` used across the 38 existing handlers, the machine-readable 402 `plan_limit` convention of `planLimitReached()` preserved verbatim, and 423/428 for the compliance gates. New endpoints add an Idempotency-Key header convention for unsafe operations, stored against the acting org with a 24-hour replay window. Paths are grouped under `/api` to sit beside their sibling routes.

POST /api/documents/upload – multipart original to object storage + seal

```

POST /api/cases/{caseId}/documents/upload
Security: cookie session (requireOrg); role: case-manageable roles
Params:  caseId (uuid, org-scoped)
Request: multipart/form-data
  file      (binary, required; max 25 MB, mime allowlist)
  notes     (string, optional)
  Idempotency-Key (header, required for retries; 24h window)
Processing:
  1. sha256(stream) server-side (same convention as documents POST)
  2. put object -> bb/org/{orgId}/case/{caseId}/doc/{docId}/v{n}
     (WORM object-lock; SSE with org DEK)
  3. insert documents row {hash, size, mime, object_key, version,
     previous_id} -> existing mismatch + metaRisk flow unchanged
  4. enqueue forensics job (OCR/EXIF/timeline checks -> metaRisk)
201 {document: {id, hash, version, metaRisk, hashMismatch, objectKey}}
402 {error:"plan_limit", limit:"maxDocsPerCase", ...} <- planLimitReached
404 case not in org | 413 too large | 415 mime rejected | 409 replay conflict

```

GET/POST /api/rag – retrieval and lawyer-admin ingestion

```

GET /api/rag/search
Security: cookie session (requireOrg)
Query:  q (string, required) | law (string, optional) | topic (string, optional)
       k (int, default 8, max 25) | asOf (ISO date, default now)
Behavior: hybrid BM25 + vector over chunks; hard filters
  is_active = true AND effective_from <= asOf
  AND (effective_to IS NULL OR effective_to > asOf)
200 {query, normalizedTokens, asOf, results: [
    {provisionId, lawId, articleNo, score, matchType, verified,
     label, snippet}}} # provision ids resolve via /api/kb/resolve
Notes: results are candidates only; display passes them through
       resolveRef() – never rendered as citations directly (PRD 6.3a)

POST /api/rag/ingest
Security: cookie session + LAWYER or platform-admin role (lawyer-admin only)
Request: {sourceUri | gazetteId, lawId?, licenseRef, dryRun?: bool}
Behavior: staging fetch -> OCR queue -> article-anchored chunk ->
         embed -> index (idempotent per content hash)
202 {jobId, chunksQueued} | 403 {error:"lawyer_admin_required"}
409 {error:"license_gate", detail:"PRD 9.2 sign-off missing for source}

```

POST /api/counter-reply/v2 – assertion pipeline + persisted traces

```

POST /api/cases/{caseId}/counter-reply/v2
Security: cookie session (requireOrg); Idempotency-Key required
Request: {layer: "L1"|"L2", persist?: bool (default true)}
Behavior: 5-stage pipeline (Section 5.1); opponent citations extracted
         and validated via resolveRef(); viability applied (5.3)
200 {assertions: [{assertionId, kind, materiality,
    response: {positionAr, positionEn, basisAr, basisEn,
              legalBasis: [{provisionId, ref, verified, label}],
              evidenceRefs: [{docId, hash}],
              traceId, needsHumanReview,
              misstatedCitations: [{ref, reason}]}}],
    reevaluatedFrom?: [provisionId]}
402 plan_limit (metered like analysis.run) | 404 | 428 (L2 without ack)

```

POST /api/tenant/keys/rotate – envelope key rotation

```

POST /api/tenant/keys/rotate
Security: cookie session; OWNER role only; platform-admin for shredding
Request: {orgId (must equal session org or explicit admin grant),
  mode: "rotate" | "retire" | "status"}
rotate: wrap new DEK version; re-wrap; lazy re-encrypt job enqueued
retire: block until re-encrypt drain; then destroy old DEK in KMS
status: {keyId, state, versions:[...], reencryptProgress}
200 {keyId, state} | 403 {error:"owner_required"}
Audit: keys.rotate / keys.retire entries via audit() (never shredded)

```

Two conventions deserve emphasis because they bind the new endpoints to existing behavior. First, the 402 plan_limit body of planLimitReached() (limit, plan, max, upgradeTo, message) is reused as-is for metered new endpoints — retrieval and counter-reply v2 count against plan capacity, never against safety features. Second, idempotency: POST endpoints that create resources (upload, ingest, counter-reply persist, key rotate) accept Idempotency-Key and return the original response on replay with 409 reserved for conflicting reuse of a key with a different payload — the same guarantee recordBillingEvent() already provides for Stripe event ids via the unique ref index (src/lib/db.ts).

8. Deliverables and Sequencing

8.1 Phase A — Platform: Postgres, RLS, envelope crypto, object storage

Phase A is the platform swap with zero user-visible behavior change; the 98 existing checks pass against both drivers at the end of the phase.

Workstream	Contents	Rollback	Estimate
Postgres + adapter	PostgresAdapter behind StorageAdapter; prisma/schema.prisma brought to parity; dual-driver CI; copy migration script with checksum verification.	Feature-flagged driver; SQLite remains default until cutover sign-off.	2 eng-weeks
RLS hardening	Policies on all tenant tables; SET LOCAL app.org_id per transaction; BYPASSRLS migration role; leakage suite T-01..T-22 green.	RLS disabled via migration revert; app layer unchanged.	1.5 eng-weeks
Envelope crypto	org_keys table; Vault Transit integration (SoftHSM dev); field encryption on the Section 3.1 scope; rotation + retirement jobs; audit entries.	Plaintext columns retained during dual-write window; drop-encryption migration reverts.	2 eng-weeks
Object storage	MinIO deployment; multipart upload path on documents route; WORM lock; forensics worker (pg-boss) + OCR baseline; PA checks.	Upload path optional; hash-only flow preserved.	2 eng-weeks

Table 8-1 — Phase A work breakdown

8.2 Phase B — Corpus: RAG ingestion, retrieval, catalogue migration

Phase B stands up the legal-content layer. The licensing gate (PRD 9.2 / 6.3a) is a precondition for bulk ingestion, not a parallel workstream — the pipeline runs against manually verified seed material until the gate opens. Workstreams: (1) ingestion service with article-anchored chunking and amendment linkage (Section 2.3); (2) embedding worker with the chosen model (Section 2.4) and pgvector HNSW index; (3) Retrieval API with temporal filtering and resolveRef-bound output (Section 2.5), replacing the in-memory BM25 path behind a flag while `/api/search` keeps its contract; (4) rule catalogue migration with golden-file parity and the `RULES_SOURCE` flag (Section 4.2); (5) interruption/suspension handling in Deadline Radar v2 (Section 4.3). Rollback per workstream is flag-based: retrieval falls back to lexical-only, catalogue falls back to compiled TS rules, and the Deadline Radar change is behind a catalogue-field presence check (empty interruption rules produce today's behavior).

8.3 Phase C — Intelligence: counter-reply v2 and the LLM service

Phase C ships the assertion-extraction pipeline with persistence and traces (Section 5), the export-blocking rule and intake safety gate (Section 6), and the self-hosted LLM service for Layer-2 language with per-sentence grounding (Section 2.6). Sequencing inside the phase matters: the deterministic pipeline ships first and becomes the Layer-1 floor; the LLM is introduced only as an optional Layer-2 renderer whose output is rejected unless every sentence carries a trace or an unverified mark. Rollback: counter-reply v2 endpoints are versioned (`/v2`) so the legacy template responses remain available; the LLM service can be disabled without touching the deterministic path.

8.4 e2e extension to 162 checks (target 160+)

The suite grows from 98 to 162 checks; the existing 98 pass unchanged in every phase. New checks are numbered by suite: T (tenant isolation, Section 3.3), PA (platform), PB (corpus), PC (intelligence).

1. **PA-01..PA-12 (Phase A, 12):** PA-01 document upload round-trip (object exists, hash matches row); PA-02 WORM lock rejects overwrite/append; PA-03 deleted-object recovery impossible within retention; PA-04 DEK wrap/unwrap round-trip; PA-05 rotation re-wraps without downtime; PA-06 retired key destroys old version after drain; PA-07 encrypted field unreadable with destroyed DEK (shred verification); PA-08 7-day deletion transitions tombstone to purge; PA-09 audit rows survive org shred; PA-10 RLS direct SQL cross-org SELECT returns zero rows (T-21/22 in CI); PA-11 PITR restore to 5 minutes before a seal restores consistent state; PA-12 dual-driver parity — 98 checks green on Postgres.
2. **PB-01..PB-16 (Phase B, 16):** PB-01 gazette fixture ingests to article-anchored chunks (article numbers preserved); PB-02 amendment article creates supersedes linkage; PB-03 retrieval with `asOf` before amendment returns only the old version; PB-04 superseded article never retrieved at `asOf=now`; PB-05 deactivated (`isActive=false`) article never retrieved; PB-06 hybrid fusion ranks exact-article matches above generic mentions; PB-07 Arabic tokenizer parity (bare-form/definite-form matching) on the new corpus; PB-08 retrieval ids all resolve via `/api/kb/resolve` (no invented refs); PB-09 unresolved candidate discarded/badged, never rendered; PB-10 catalogue migration golden-file parity on 5 fixtures; PB-11 catalogue edit by non-lawyer rejected (403); PB-12 catalogue version chain retains history; PB-13 interruption rule tolls deadline correctly in a recess scenario; PB-14 counterargument renders when

present; PB-15 embedding worker health + reindex idempotency; PB-16 lexical fallback serves when vector service is down.

3. **PC-01..PC-14 (Phase C, 14):** PC-01 segmentation fixtures (5 Arabic pleadings) segment as expected; PC-02 classification precision on fixture set at or above threshold; PC-03 materiality ordering matches expected point order; PC-04 opponent citation extraction returns correct {lawId, article} pairs; PC-05 misstated citation flagged with neutral wording; PC-06 unresolved opponent citation never rendered as citation; PC-07 every response line joins to at least one trace row; PC-08 per-point evidence hash present (AC3); PC-09 missing predicate auto-downgrades relevance; PC-10 high relevance with missing predicates forces needsHumanReview; PC-11 export blocked without lawyer review (423 lawyer_review_required); PC-12 DRAFT-labeled export carries watermark + acknowledgment snapshot; PC-13 LLM sentence without grounding marked unverified and badged; PC-14 external-LLM payload scanner proves no case facts in outbound anonymized-rule requests.

Counting: 98 existing + 22 isolation + 12 + 16 + 14 = **162 checks**, meeting the 160+ target with margin. Each phase's checks are merged at phase exit, and the README's compliance matrix is updated in the same commit so documentation never drifts from the suite.

9. Decision Register — Only Humans Can Make These

The following decisions are outside engineering authority. Each is stated with the options on the table and the section of this specification it gates. They should be resolved in the order listed, because earlier decisions unblock phase work.

1. **Embedding-model hardware budget.** bge-m3 with a GPU (recommended for retrieval quality) versus multilingual-e5-large on CPU-only hosting (Section 2.4). Determines the Phase B infrastructure line item and the vector-store corollary (pgvector first, Qdrant if scale demands).
2. **LLM model, GPU sizing, and license verification.** Qwen2.5-32B/72B-Instruct versus an Arabic-specialist family; AWQ quantization and serving capacity; license review for the selected weights (Section 2.6).
3. **Lawyer validation pipeline.** Recruitment of 2-3 licensed Kuwaiti lawyers (civil, criminal, family), hours and headcount for the Phase 0 citation audit, the go/no-go rule the PRD already imposes (no advisors, no ground-relevance scoring or draft generation), and ownership of the rule-catalogue validation workflow (Sections 4.3, 9 of the PRD).
4. **MoJ / Official Gazette data access and licensing.** The bulk-ingestion gate of PRD 9.2 and 6.3a: MoU or manual-entry-first for launch; which sources are reproducible and under what terms (Section 2.3).
5. **Data residency and hosting region.** Confirmation of the MOJ/CITRA posture for case data, resolution of the PRD 6.3a self-hosting judgment call (C4) including whether the legal-content layer may ever touch external APIs, and procurement of the hosting environment (Sections 2.1, 2.6).

6. **KMS custody and audit-trail encryption split.** HashiCorp Vault operational ownership, master-key ceremony custody, and counsel sign-off that audit_logs live outside the org envelope (Section 3.1) so crypto-shredding cannot erase compliance history.
7. **Admission-category policy.** Whether the counter-reply response categories should ever include admission/qualified admission (PRD 5.6 AC2) or whether the current conservative deviation (never suggest admitting) is ratified as product policy (conflict C5).
8. **Appeal-window validation sign-off.** The WINDOWS values in the catalogue (civil 40 / criminal 20 / family 30) require Phase 0 lawyer validation with gazette references — the family window is explicitly flagged "under verification" in the engine today (conflict C6).
9. **Retention schedule.** The retention period applied to WORM-locked evidence objects and the interplay with the 7-day deletion workflow and Kuwaiti counsel's advice on evidence preservation duties (Sections 2.7, 3.4).